Durée	**Matériel**	**Conditions**	**Sujet**
4 périodes	-	Travail individuel	Commandes de base PowerShell

Contexte de l'exercice

Cet exercice va vous permettre d'explorer quelques commandes de base de PowerShell. Parfois, des **suppléments théoriques** figurent dans les encadrés jaunes : **lisez-les** avant de commencer !

Conseil : **n'utilisez pas ChatGPT** pour faire ces exercices, vous n'apprendrez rien…

https://learn.microsoft.com/fr-fr/powershell/?view=powershell-5.1

Version de PoweShell

Ouvrez une invite de commande PowerShell et cherchez la version de PowerShell installée sur votre système :

- Au moyen de la cmdlet « Get-Host »
- Au moyen de la variable « $PSVersionTable »

AutoComplete
Observez qu'en tapant « Get-H » suivit d'un Tab, la console PowerShell complète la ligne pour lister toutes les cmdlet qui commencent par « Get-H …»

Résultat des deux méthodes :

```
Name            : ConsoleHost
Version         : 5.1.26100.7705
InstanceId      : 7be0f351-c42c-47b7-8b7e-ad838c31dc7a
UI              : System.Management.Automation.Internal.Host.InternalHostUserInterface
CurrentCulture  : fr-CH
CurrentUICulture : fr-FR
PrivateData     : Microsoft.PowerShell.ConsoleHost+ConsoleColorProxy
DebuggerEnabled : True
IsRunspacePushed : False
Runspace        : System.Management.Automation.Runspaces.LocalRunspace
```

```
Name                      Value
----                      -----
PSVersion                 5.1.26100.7705
PSEdition                 Desktop
PSCompatibleVersions      {1.0, 2.0, 3.0, 4.0...}
BuildVersion              10.0.26100.7705
CLRVersion                4.0.30319.42000
WSManStackVersion         3.0
PSRemotingProtocolVersion 2.3
SerializationVersion      1.1.0.1
```

PowerShell, un langage orienté objet

Tout comme Java, PowerShell est un langage de programmation orienté objet.

Les cmdlets de PowerShell sont des petits programmes qui exécutent des fonctions bien spécifiques. Leur nom est constitué d'un verbe et d'un nom séparés par un tiret. Elles peuvent être suivies de paramètres.

La principale chose à retenir sur les cmdlets est **qu'ils retournent des objets**.

Essayez donc de trouver la date et l'heure actuelle en assignant le résultat dans une variable :

```
get-date
```

Et maintenant, que peut-on trouver comme informations dans cet objet ? Cherchez par vous-même…

Pour accéder aux attributs d'un objet, utilisez le « . »

```
(get-date).AddDays(10)        (get-date).Month
```

Le caractère « Pipe »

Le caractère « | » permet de passer le résultat d'une commande ou un objet à une autre commande.

Par exemple, pour trier un tableau :

```
$tableau | Sort-Object
```

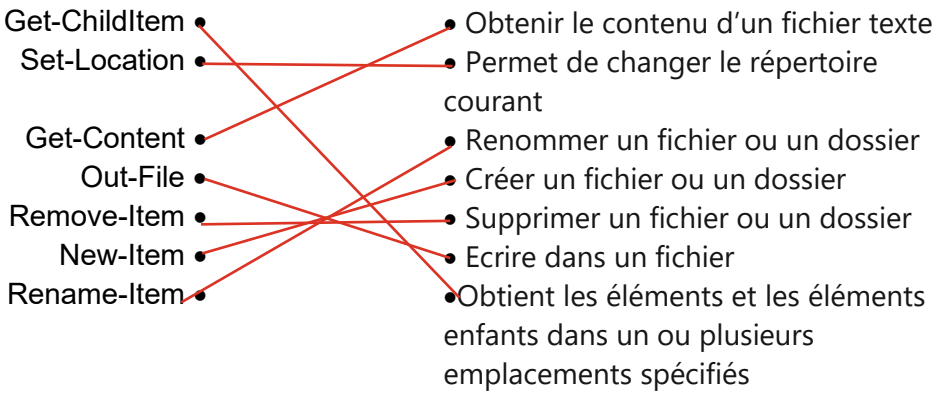
A vous de jouer, écrivez la date et l'heure actuelle dans un fichier nommé « maDate.txt »

Pour écrite dans un fichier, utilisez la cmdlet « Out-File »

get-date| out-file "\\stu.net.fr.ch\perso$\Users\BangerterA\Desktop\date.txt"

Travailler avec le système de fichiers

PowerShell permet de travailler de manière très simple avec le système de fichier. **Reliez** les cmdlet avec leur utilisation :



Exercice pratique : récupérez la liste de tous les dossiers présents dans votre disque P:\ et triez-les de Z à A :

Get-ChildItem -path P:\ -name | Sort-Object Name -Descending

Paramètres

La plupart des cmdlets supportent le paramètre « -InputObject » qui permet de spécifier le ou les objets qui vont être traités par le cmdlet. Par exemple :

```
$a = 1..5
Out-Host –InputObject $a
```

Que fait le code ci-dessus ? Cherchez à comprendre chacun des éléments..

$a= 1..5 ça crée un tableau qui va de 1 à 5 (le .. pourait signifier à)
et le Out-Host affiche le contenu de $a

Chaque cmdlet a ses propres paramètres selon sa fonction. Vous les trouverez dans la documentation

Également toutes les commandes qu'on peut utiliser avec un « | », doivent avoir ce paramètre -InputObject. Ici Out-Host peut donc être utiliser avec « | ». Par exemple :

```
$a = 1..5
$a | Out-Host
```

Les opérateurs

Le tableau ci-dessous présente les opérateurs les plus souvent utilisés :

Les opérateurs de comparaison PowerShell ne ressemble pas à ce que vous connaissez déjà…

Arithmétique	
+	Addition
-	Soustraction
*	Multiplication
/	Division
%	Modulo (reste de la division)
Comparaison	
-eq	Egal à (equal)
-lt	Plus petit que (less than)
-gt	Plus grand que (greater than)
-ge	Plus grand ou égal (greater or equal)
-le	Plus petit ou égal (less or equal)
-ne	Pas égal à (not equal)
Binaires	
-not	Non
-and	Et
-or	Ou
Correspondance	
-like	Correspond à une wildcard
-notlike	Ne correspond pas à une wildcard
-contains	Une collection contient la valeur

Maintenant, remplissez le tableau ci-dessous **sans ordinateur** !!

Code	Résultat	Vérification
5 + 9	14	
`"test" -eq "TesT"`	true	true car -eq ne prend pas en compte les majuscules
50 -lt 60	true	true
50 -eq "`cinquante`"	false	on peu pas comparer un string a un int
(60 -ne 50) -or ($true)	true	true
(60 -ne 50) -and ($true)	true	et logique
`"test" -like "t*t"`	false	true
`1..50 -contains 10`	true	true

Une fois que c'est fait, **contrôler** vos réponses au moyen de l'ordinateur et remplissez la deuxième colonne.

Les objets…

La cmdlet « Get-Member » vous permet de :

- Savoir quel est le type (la classe) de l'objet
- Connaitre toutes les méthodes et attributs disponibles pour cet objet

```
$a = "2"
$b = 2
$c = 1..5
```

Quel et le type de $a , $b et de $c ?

$a string
$b int32
$c int32

```
$a -eq $b
```

Que retourne le test ci-dessus ? Pourquoi ?

true car meme si $a est un string c'est un nombre dedans

Les tableaux et autres collections

PowerShell gère de manière très simple et puissante les tableaux. Voici quelques exemples…

Les variables peuvent être de type « Array » (tableau). Pour les tableaux de base, on peut adresser chaque membre par son index sous forme d'entier :

```
$a = 1..5
$a[1] = 9
```

Pour créer un tableau vide :
$a = @()

Que contient le tableau $a ?

1 9 3 4 5

```
$a = 1..5
$a[1] = "une chaine"
```

Et maintenant ?

1 une chaine 3 4 5

Surpris du résultat ? Un tableau est un tableau d'objet, peu importe le type d'objet que vous y mettez... A vous de faire attention !

On peut aussi faire des tableaux associatifs (HashTable), dont les index sont des noms au lieu d'entiers

Pour créer un tableau associatif vide :
$a = @{}

```
$a = @{}
$a["student"] = 1
$a["teacher"] = 1
$a["student"] = $a["student"] + 15
```

Que retourne « $a["student"] » ? Et « $a.student » ?

les deux retournent 16

Les boucles

Toutes les boucles que vous connaissez existent également en PowerShell. Une qui est très utile est la boucle **foreach** pour traiter tous les éléments d'un tableau.

En voici un exemple :

```
$classe = @("Tom","Clara","Noé","Sarah")
foreach($eleve in $classe)
{
      Write-Host $eleve
}
```

foreach** est un alias de la cmdlet **forEach-Object

Et maintenant, **à vous de jouer** ! Ecrivez la boucle qui permet de savoir quels fichiers/dossiers contiennent la lettre « A »

```
$fichiers = Get-ChildItem P:\
…
```

```
[ ]Videos
[ ]system
[ ]Pictures
[ ]Music
[ ]WINDOWS
[X] Application Data
[X] AppData
[X] NetApp
[ ]Documents
[X] Downloads
[ ]Desktop
[ ].gitconfig
```

Une fois terminé, montrez votre solution à votre enseignant

```
$fichiers = get-childitem -path c:\ -name
foreach($fichier in $fichiers){
if($fichier -like "*a*"){
write-host "[X]" $fichier
}
else{
write-host "[ ]" $fichier
}
}
```

Environnement de développement

A présent, que vous maitrisez la ligne de commande PowerShell, il faut utiliser un environnement de développement un peu plus évolué…

Lancez donc VSCode et **installez** les extensions officielles « PowerShell ».

Reprenez le code écrit à l'exercice précédent et **enregistrez** le dans un fichier « script PowerShell ».

*Les scripts PowerShell ont l'extension **.ps1***

Exécutez votre script depuis VSCode. Le résultat doit être le même que celui obtenu plus haut, bien évidemment…